_ TELECOM ESCUELA
TÉCNICA VLC SUPERIOR
DE INGENIERÍA DE
TELECOMUNICACIÓN

UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

GRADO EN TECNOLOGÍA DIGITAL Y MULTIMEDIA

ASIGNATURA: INTERACCIÓN, SENSORES Y TRANSDUCTORES

MEMORIA TRABAJO FINAL:

CONTROLADOR DE OBJETOS EN BLENDER CON M5STACK Y OSC

CURSO 2024/2025

PAULA OTERO PAÑOS

ANNA REIG BORRÁS

Índice

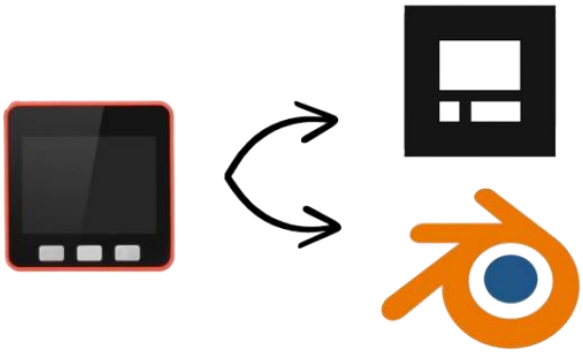
1. INTRODUCCIÓN 1
2. OBJETIVOS 2
3. DESARROLLO 2
3.1. M5Stack 3
3.2. TouchOSC 7
3.3. Blender 9
3.4. Ejecución 19
4. CONCLUSIONES 21

1. INTRODUCCIÓN

El desarrollo de sistemas interactivos que combinan hardware y software es un pilar fundamental en la exploración de tecnologías aplicadas, como sensores y transductores. En este trabajo final de la asignatura se aborda la creación de un sistema que aprovecha los conceptos y herramientas adquiridos durante el curso, implementando un enfoque práctico y funcional.

La idea para este proyecto nació inspirada en los videojuegos donde es común encontrar la posibilidad de personalizar a tu personaje antes de iniciar el juego. Esta interacción inicial nos pareció un concepto fascinante, y aunque decidimos llevar a cabo una versión más reducida y enfocada, consideramos que este proyecto puede servir de base para desarrollos más complejos y ambiciosos en el futuro.

El sistema desarrollado utiliza el dispositivo M5Stack-FIRE como base de hardware, integrada con Blender mediante el protocolo de mensajes OSC (Open Sound Control). Este enfoque permite la interacción dinámica entre el entorno físico y el entorno digital, proporcionando una experiencia de control enriquecida y versátil. Además, se ha incorporado TouchOSC como herramienta gráfica de visualización, que facilita la interacción en tiempo real entre el M5Stack y Blender.



El objetivo principal de este proyecto es diseñar e implementar un sistema que combine la capacidad de detección y procesamiento del M5Stack con la potencia gráfica y la personalización de Blender, logrando una solución que ilustre de manera práctica las aplicaciones de los sensores y transductores. A lo largo del desarrollo, se abordan aspectos importantes como la comunicación OSC, la integración de periféricos, la programación embebida y el diseño de interfaces visuales.

El proyecto no solo busca demostrar la funcionalidad técnica del sistema, sino también explorar los desafíos y las soluciones en el proceso de desarrollo. De este modo, se presenta una experiencia integral que conecta los conocimientos teóricos adquiridos en la asignatura con su aplicación en un entorno real y creativo.

2. OBJETIVOS

El objetivo principal de este trabajo es implementar un sistema interactivo que integre el controlador M5Stack-FIRE con Blender mediante mensajes OSC, utilizando Touch OSC como herramienta de visualización gráfica. Este proyecto busca demostrar la funcionalidad de los componentes seleccionados y su capacidad para trabajar en conjunto, mientras se optimizan los aspectos de comunicación, control e interacción.

Para lograr este objetivo general, se plantean los siguientes objetivos específicos:

- **Diseñar e implementar la comunicación** OSC entre el M5Stack-FIRE y Blender, permitiendo la transferencia de datos en tiempo real para el control dinámico del entorno gráfico.
- **Configurar y personalizar TouchOSC** como una interfaz gráfica que represente de manera clara y precisa las acciones realizadas por el M5Stack, mejorando la experiencia del usuario.
- **Integrar sensores y periféricos compatibles con el M5Stack-FIRE** para ampliar las capacidades del sistema, facilitando la captura de datos relevantes para la interacción.
- **Desarrollar la lógica de programación embebida** necesaria para procesar las entradas de los sensores y generar comandos adecuados que se envíen a Blender mediante OSC.
- **Evaluar el rendimiento y la funcionalidad del sistema** mediante pruebas prácticas, asegurando una comunicación fiable entre todos los componentes y una experiencia fluida para el usuario.
- **Documentar el proceso de desarrollo**, incluyendo diagramas, pseudocódigo, esquemas de conexión y pruebas de funcionamiento, para garantizar una comprensión integral del proyecto.

3. DESARROLLO

En esta sección, se detalla el proceso de implementación del sistema interactivo que integra el controlador M5Stack-FIRE con Blender a través del protocolo de mensajes OSC. Desde la configuración inicial de los componentes hasta la integración de TouchOSC como interfaz gráfica adicional, se abordan los pasos necesarios para lograr una comunicación efectiva y fluida entre los elementos del sistema.

A lo largo de este apartado, se describen las decisiones de diseño adoptadas, las estrategias de programación utilizadas y las soluciones aplicadas para resolver los desafíos técnicos encontrados durante el desarrollo. Además, se presenta una

descripción detallada de la configuración de hardware y software, los métodos de comunicación implementados y las pruebas realizadas para validar el funcionamiento del sistema.

Este enfoque permite mostrar el proceso completo de creación, desde la conceptualización inicial hasta la ejecución práctica, destacando los aspectos más relevantes del proyecto y su relación con los objetivos planteados.

3.1. M5Stack

La implementación del M5Stack como componente principal para interactuar con Blender y TouchOSC fue un proceso meticuloso que requirió varias etapas de desarrollo y solución de problemas. La primera tarea consistió en establecer la capacidad del M5Stack para enviar mensajes OSC. Para ello, fue necesario descargar la carpeta `uosc` desde el enlace (https://github.com/SpotlightKid/micropython-osc) proporcionado en la práctica 4 del M5Stack-FIRE. Esta carpeta, extraída del repositorio GitHub de la librería `micropython-osc`, se colocó en la ruta específica del entorno de simulación: `m5stack-simu-pyqt\app`.

Con la configuración inicial completada, el primer paso práctico fue probar un código básico que validara la conexión entre el M5Stack y TouchOSC (sacado de la misma práctica). Una vez verificada la comunicación, se procedió a extender la funcionalidad para incluir a Blender en el circuito. El establecimiento exitoso de esta conexión fue un acontecimiento significativo, ya que permitía que los mensajes OSC enviados desde el M5Stack fueran interpretados y procesados tanto por TouchOSC como por Blender (se explicará más en profundidad estos primeros pasos en el apartado de 'Blender').

A continuación, se trabajó en el mapeo de los botones físicos del M5Stack para ejecutar acciones específicas en Blender. Se asignaron funciones claras a cada botón: el botón A controlaba el cambio de color del objeto (`/color`), el botón B ajustaba su tamaño (`/size`), y el botón C permitía rotarlo (`/rotate`). Estas funciones se fueron desarrollando y añadiéndoles complejidad, como se explicará más adelante. Durante este proceso, se utilizó un código inicial sencillo para probar cada funcionalidad por separado. Esta metodología permitió aislar problemas y asegurarse de que cada componente del sistema operar de manera independiente antes de integrarlos.

Como complemento a esta funcionalidad, se añadió una interfaz gráfica en la pantalla de la M5Stack. Este mensaje describe claramente el propósito de cada botón, mostrando instrucciones como '*btnA: modo color*', '*btnB: modo tamaño*' y '*btnC: modo girar*'. Esto mejora no solo facilitó la comprensión del funcionamiento

del sistema para el usuario, sino que también proporcionó una referencia visual clara para interactuar con los controles físicos del dispositivo.

Figura 1. Interfaz gráfica del M5Stack

Tras el correcto funcionamiento de los botones, el enfoque se trasladó a utilizar los datos del giroscopio del M5Stack (yaw, pitch y roll) como argumentos dinámicos para modificar el estado del objeto en Blender. El eje `yaw` se empleó para cambiar el color del objeto (`/color`), el `pitch` para controlar su rotación (`/rotate`), y el `roll` para ajustar su tamaño (`/size`). Esto paso implicó enviar valores en tiempo real desde el M5Stack, lo que añadió una capa de complejidad al proceso, ya que requería que Blender interpretara estos datos de manera fluida y sin interrupciones.

Adicionalmente, se introdujo la funcionalidad de reinicio (`/reset`), que se ejecutaba automáticamente cada vez que se presionaba cualquiera de los botones. Esto aseguraba que el objeto volviera a un estado inicial predeterminado antes de aplicar nuevas modificaciones, lo que facilitó las pruebas y mejoró la coherencia del sistema.

Por último, se añadió el envío de mensajes OSC a TouchOSC. Aunque este paso resultó desafiante debido a problemas iniciales de configuración, finalmente se logró establecer la comunicación efectiva. Esto permitió que TouchOSC actuara como la interfaz gráfica de las acciones realizadas por el M5Stack.

Para mejorar la experiencia y evitar transiciones bruscas, se implementó una normalización de los valores del giroscopio. Los datos de yaw, pitch y roll, originalmente en rangos negativos y positivos, se transformaron para estar en un intervalo entre 0 y 1. Esto garantizó un cambio más suave y realista en las características del objeto, eliminando los saltos abruptos que podían ocurrir al pasar de un valor negativo a uno positivo.

A resumidas cuentas, el desarrollo de la funcionalidad del M5Stack fue un proceso iterativo que implicó diversas pruebas y ajustes. Desde la configuración inicial de la comunicación OSC hasta la implementación de funcionalidades avanzadas como la normalización de valores y el manejo de botones, cada etapa contribuyó a construir un sistema robusto y funcional. Los problemas encontrados, como la falta

de comunicación inicial o los cambios bruscos en los valores del giroscopio, se resolvieron mediante soluciones prácticas que optimizaron el rendimiento general del sistema. El resultado final fue una integración completa que permitió al M5Stack controlar de manera precisa y dinámica las acciones del objeto en Blender, con una representación clara en TouchOSC.

Pseudocódigo:

```
# Conexión a la red WiFi
Conectar a la red WiFi con SSID y contraseña
Si la conexión es exitosa:
    Imprimir "Conectado con IP" y la IP del dispositivo
Si no:
    Imprimir "No conectado"

# Configuración de conexión OSC
Establecer IP del servidor y puerto para OSC
Crear un socket UDP para enviar mensajes OSC
Configurar cliente OSC para TouchOSC con IP del servidor y puerto TouchOSC

# Configuración de la interfaz gráfica
Mostrar en pantalla:
    - "Nombre del controlador"
    - Funciones de los botones:
        Botón A -> "Modo color"
        Botón B -> "Modo tamaño"
        Botón C -> "Modo girar"

# Inicialización de variables y sensores
Inicializar estados de los botones como None
Inicializar el sensor IMU
Establecer el modo actual como None

# Función para enviar mensajes OSC
Definir send_osc_message(dirección, mensaje):
    Crear el mensaje OSC combinando dirección y mensaje
    Enviar el mensaje al servidor OSC usando el socket UDP
    Imprimir "Mensaje enviado:" y el mensaje

# Función para reiniciar el estado del objeto
Definir reset_object():
    Llamar a send_osc_message con "/reset" y "reset"

# Función para verificar los botones
Definir check_buttons():
    Si el botón A está presionado y su estado no es "pressed":
        Cambiar el modo actual a "color"
```

```
        Llamar a reset_object
        Cambiar el estado del botón A a "pressed"
    Si el botón A no está presionado y su estado no es "released":
        Cambiar el estado del botón A a "released"

    Si el botón B está presionado y su estado no es "pressed":
        Cambiar el modo actual a "size"
        Llamar a reset_object
        Cambiar el estado del botón B a "pressed"
    Si el botón B no está presionado y su estado no es "released":
        Cambiar el estado del botón B a "released"

    Si el botón C está presionado y su estado no es "pressed":
        Cambiar el modo actual a "rotate"
        Llamar a reset_object
        Cambiar el estado del botón C a "pressed"
    Si el botón C no está presionado y su estado no es "released":
        Cambiar el estado del botón C a "released"

# Función para actualizar según el movimiento
Definir update_based_on_movement():
    Obtener yaw, pitch y roll del sensor IMU
    Normalizar los valores:
        - yaw entre 0 y 1
        - pitch entre -90 y 90 ajustado entre 0 y 1
        - roll entre -90 y 90 ajustado entre 0 y 1

    Si el modo actual es "color":
        Enviar mensaje OSC con "/color" y el valor de yaw
        Enviar mensaje a TouchOSC con "/color" y el valor normalizado de
yaw
    Si el modo actual es "size":
        Enviar mensaje OSC con "/size" y el valor de roll
        Enviar mensaje a TouchOSC con "/size" y el valor normalizado de
roll
    Si el modo actual es "rotate":
        Enviar mensaje OSC con "/rotate" y el valor de pitch
        Enviar mensaje a TouchOSC con "/rotate" y el valor normalizado de
pitch

# Bucle principal
Mientras el programa está en ejecución:
    Llamar a check_buttons para verificar el estado de los botones
    Si hay un modo activo:
        Llamar a update_based_on_movement para actualizar datos en base al
movimiento
    Esperar 0.1 segundos
```

3.2. TouchOSC

El uso de TouchOSC tuvo como finalidad principal ofrecer una interfaz gráfica que permitiera visualizar de forma clara las acciones realizadas por el M5Stack en Blender. Aunque en un principio se intentó utilizar TouchOSC como un intermediario directo entre el M5Stack y Blender, los múltiples intentos realizados no dieron los resultados esperados. Tras consultar con el profesor, se decidió reconfigurar su propósito, empleándolo exclusivamente como una herramienta de visualización. Este cambio de enfoque simplificó el desarrollo y permitió centrar los esfuerzos en optimizar la representación gráfica de las acciones.

El diseño de la interfaz comenzó con la incorporación de tres controladores deslizantes o *faders* en TouchOSC, representando las principales acciones que el M5Stack realiza sobre el objeto en Blender: cambiar el color, modificar el tamaño y ajustar la rotación. Cada uno de estos controladores fue nombrado según la acción que simboliza: `color`, `size` y `rotate`. Esta personalización no solo ayudó a organizar la interfaz de manera lógica, sino que también facilitó la asociación entre los elementos gráficos y los mensajes OSC procesados en Blender.

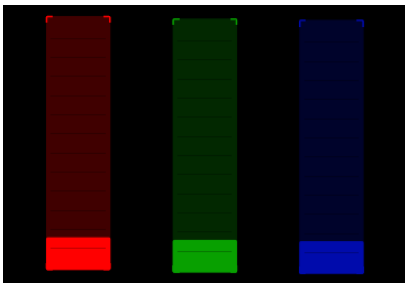


Figura 2. Interfaz gráfica de TouchOSC

La configuración fue un paso esencial para garantizar su correcto funcionamiento. Se utilizó el protocolo UDP para establecer la conexión, configurando el host en `127.0.0.1`, ya que tanto el M5Stack como TouchOSC operaban en el mismo dispositivo durante las pruebas. En cuanto al puerto de recepción, se asignó el puerto en el que el cliente conectado a TouchOSC debía recibir los mensajes. La configuración del puerto de envío se dejó vacía, dado que en este caso TouchOSC no actuaba como un emisor de mensajes. También, se dejó el parámetro Zeroconf en su configuración predeterminada.

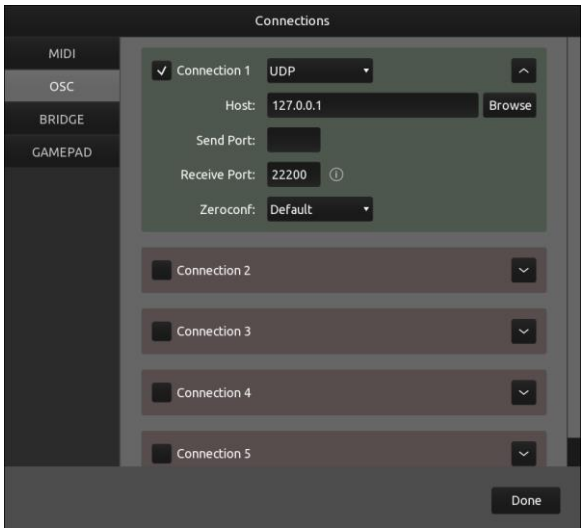


Figura 3. Configuración de TouchOSC

Durante el desarrollo, surgieron varios problemas. Uno de los principales fue la dificultad inicial para configurar TouchOSC como un intermediario entre el M5Stack y Blender. Este enfoque resultó ser demasiado complejo y no logró establecer una comunicación estable. Consultar con el profesor fue determinante para cambiar la estrategia y optar por usar TouchOSC únicamente como una herramienta de visualización. La decisión final fue establecer conexión solamente con el M5Stack. Otro problema se presentó al configurar los puertos de conexión, ya que en un principio no estaba claro cómo debía asignarse el puerto de recepción para garantizar que los mensajes enviados por la M5Stack fueron procesados correctamente. Este problema se resolvió ajustando los parámetros para alinear el puerto de recepción con el cliente de TouchOSC.

El resultado final de esta integración fue una interfaz gráfica funcional y efectiva. Los controles deslizantes de TouchOSC permitieron visualizar en tiempo real las acciones ejecutadas por el M5Stack en Blender, mostrando cómo cambiaban el color, el tamaño y la rotación del objeto. Este sistema no solo mejoró la comprensión del funcionamiento global del proyecto, sino que también facilitó la depuración, ya que era posible verificar de forma visual que los mensajes OSC enviados desde la M5Stack eran interpretados correctamente por Blender.

Por lo que, aunque inicialmente se concibió con un propósito más ambicioso, TouchOSC demostró ser una herramienta importante como interfaz gráfica. Su configuración y diseño no solo permitieron representar de manera clara las interacciones entre el M5Stack y Blender, sino que también simplificaron el proceso de validación del sistema. Su uso en este proyecto consolidó su utilidad como un complemento efectivo para visualizar y analizar las funcionalidades implementadas.

3.3. Blender

El desarrollo de la funcionalidad en Blender comenzó con la necesidad de integrar el manejo de mensajes OSC para que el programa pudiera interactuar con el M5Stack y TouchOSC. Este proceso inició con la instalación de un complemento adicional, específicamente el add-on NodeOSC. Para lograr esto, fue necesario descargar el archivo comprimido `NodeOSC_241.zip` desde su repositorio en GitHub (https://github.com/maybites/NodeOSC/releases/tag/V2.4.1) y colocarlo en la carpeta correspondiente al directorio de Blender: **C:\Users\paula\AppData\Roaming\BlenderFoundation\Blender\4.2\scripts\addons.**

En nuestro caso, no existía esa carpeta, por lo que hubo que crearla manualmente y depositar en ella el archivo descomprimido.

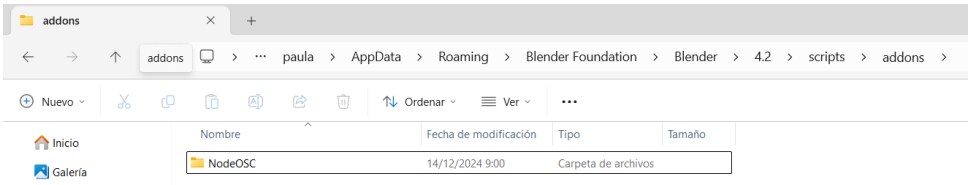


Figura 4. Ruta del add-on en Blender.

Posteriormente, el complemento se activó desde el menú de preferencias en Blender, bajo la sección de Add-ons, seleccionando 'System: NodeOSC'. Este paso fue fundamental para habilitar la capacidad de Blender de recibir y procesar mensajes OSC.

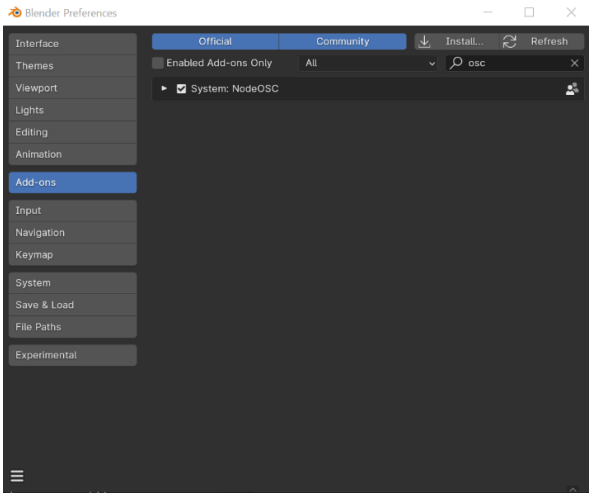


Figura 5. Panel de preferencias

Una vez instalado y configurado el add-on, se procedió a probar la funcionalidad de un script básico (que se ha explicado anteriormente) diseñado para manejar mensajes OSC enviados desde el M5Stack. Este código inicial sirvió como prueba de concepto para verificar que los mensajes fueran correctamente interpretados por Blender.

El primer objetivo de nuestro proyecto fue lograr que Blender respondiera a los mensajes enviados mediante el protocolo OSC desde un simulador del M5Stack ejecutado en Thonny. Para ello, creamos un objeto básico en Blender, específicamente una esfera, y nos propusimos que esta cambiara de color a rojo y aumentara de tamaño al presionar el botón A del M5Stack. Este objetivo inicial fue fundamental, ya que nos permitió establecer la comunicación entre el hardware y el software y comprobar que los mensajes enviados llegaban correctamente a Blender.

Para llevar a cabo esta interacción entre el M5Stack y Blender, implementamos un programa en MicroPython, ejecutado desde Thonny, que cumplía varias funciones esenciales. A continuación, se explican los detalles de cada una de ellas.

El primer paso consistió en conectar el M5Stack a una red Wi-Fi. Sin embargo, en este caso, no se trataba de una red real, sino de una red simulada específicamente creada para el proyecto. El objetivo de esta configuración era permitir el flujo de datos mediante el protocolo OSC sin depender de una conexión externa a Internet, ya que todo el proceso se llevó a cabo localmente desde nuestra casa. Más adelante, cuando probamos el sistema en el laboratorio, tuvimos que hacer algunos ajustes respecto a la IP y la red Wi-Fi, lo cual se detallará más abajo.

La red Wi-Fi "falsa", configurada con el SSID "GTDM" y la contraseña "12345678", se utilizaba únicamente como medio de simulación, asegurando que el dispositivo pudiera establecer una conexión activa para enviar mensajes OSC sin necesidad de depender de una infraestructura externa. Una vez establecida la conexión, el programa verificaba si esta era exitosa y mostraba la dirección IP asignada al dispositivo, lo cual era crucial para la comunicación local entre el M5Stack y Blender. Si la conexión fallaba, el programa notificaba el error, lo que facilitaba la identificación de problemas durante la simulación.

A continuación, configuramos los parámetros del servidor OSC, que era el encargado de recibir los mensajes. En este caso, utilizamos la dirección IP de localhost (127.0.0.1), ya que Blender se ejecutaba en la misma máquina que el simulador de M5Stack. También especificamos el puerto de recepción (9009), asegurándonos de que este estuviera correctamente configurado en el script de Blender para que pudiera recibir los mensajes enviados desde el M5Stack. Para enviar los mensajes, creamos un socket de comunicación que utilizaba el protocolo UDP, que es estándar en OSC.

El siguiente paso fue configurar el botón A del M5Stack, para que el programa pudiera detectar cuándo era presionado. Para ello, definimos el botón como una entrada digital con una resistencia pull-up, lo que garantiza que su estado sea claro (alto o bajo). Además, implementamos una variable para almacenar el estado anterior del botón, lo cual fue clave para detectar cambios en su estado, como cuándo se presionaba o se soltaba. Esta configuración permitió que el sistema detectara la interacción del usuario con precisión y diferenciara entre un botón presionado y uno liberado.

Una vez configurado el botón, diseñamos una función que enviaba mensajes OSC al servidor de Blender cada vez que se presionaba el botón A. Los mensajes OSC contenían una dirección (similar a una ruta) y un mensaje asociado, en este caso indicando que el botón A había sido presionado. Estos mensajes se enviaban a través del socket previamente configurado, y cada vez que se transmitía un mensaje, se imprimía una notificación en la consola, lo que facilitaba la depuración del sistema.

El programa verificaba continuamente el estado del botón A. Si detectaba que el botón había sido presionado por primera vez, enviaba un mensaje OSC al servidor de Blender con la instrucción correspondiente. Tras enviar el mensaje, el estado del botón se actualizaba a "presionado" para evitar el envío de múltiples mensajes mientras el botón permaneciera pulsado. Cuando el botón se liberaba, el estado se actualizaba nuevamente, pero en este caso no se enviaba ningún mensaje, ya que solo queríamos responder al evento de presionar el botón.

El programa funcionaba en un bucle continuo que verificaba el estado del botón cada 0,1 segundos. Esta pequeña pausa evitaba lecturas demasiado rápidas, lo que podría generar problemas en el envío de mensajes o en la detección precisa de los eventos del botón.

Simultáneamente al desarrollo del código para el M5Stack en Thonny, se implementó el código en Blender con el objetivo de que este pudiera recibir y procesar los mensajes OSC enviados por el M5Stack. El propósito de esta interacción era lograr que, al presionar el botón A del M5Stack, Blender respondiera cambiando el color y tamaño de una esfera en la escena 3D.

El servidor OSC en Blender estaba diseñado para escuchar continuamente en el puerto 9009, el mismo puerto configurado en el código de Thonny para el envío de los mensajes. Al recibir un mensaje, el servidor decodificaba el contenido y lo pasaba a la función que manejaba la lógica para ejecutar las acciones correspondientes en Blender. En este caso, el mensaje OSC enviado desde el M5Stack indicaba, a través de la dirección "/test", que el botón A había sido presionado. Cuando el servidor recibía este mensaje, el script de Blender respondía realizando dos acciones en la esfera: cambiaba su escala y su color.

La escala de la esfera se modificaba duplicando su tamaño, configurando las dimensiones de la esfera a (2, 2, 2), lo que causaba que la esfera se agrandara. Además de esto, el código verificaba si la esfera tenía un material asignado. Si este material existía, el script cambiaba su color a rojo utilizando el valor (1, 0, 0, 1) en formato RGBA. Si no se encontraba un material asignado, el script informaba de este hecho en la consola, lo que facilitaba la detección de errores.

El servidor OSC se ejecutaba en un hilo separado dentro de Blender, lo que permitía que la escucha de mensajes no bloqueara otras funciones de la interfaz o el proceso de renderizado. Este enfoque de multihilo es fundamental para garantizar que Blender mantuviera su rendimiento mientras recibía y procesaba los mensajes OSC. Además, para evitar que el servidor OSC sobrecargara la CPU del sistema, se incluía una pausa de 0,1 segundos en cada iteración del bucle de escucha. Esto aseguraba que las lecturas de mensajes fueran eficientes y no afectaran el rendimiento global de Blender.

El proceso completo de comunicación entre el M5Stack y Blender comenzó cuando el M5Stack detectaba que el botón A había sido presionado. El código en Thonny se encargaba de detectar esta interacción y enviaba un mensaje OSC al servidor de Blender. Al recibir este mensaje, Blender respondía cambiando tanto el color como el tamaño de la esfera, lo que confirmaba que la interacción entre el M5Stack y el software estaba funcionando correctamente.

Este enfoque simultáneo en el desarrollo del código para ambos dispositivos permitió que se pudiera probar la comunicación entre ellos en tiempo real, logrando una interacción fluida que servía como base para futuras mejoras y expansión del sistema. Al final, el resultado demostró que el flujo de mensajes OSC entre el M5Stack y Blender estaba correctamente establecido, lo que abrió la puerta a la implementación de interacciones más complejas en etapas posteriores del proyecto.

A continuación, se adjunta el pseudocódigo inicial que se pensó para ambos programas y unas capturas de pantalla del resultado logrado tras esta primera prueba.

Pseudocódigo inicial en Blender:

```
Iniciar servidor OSC:
    Definir dirección IP --> en este caso pondremos "127.0.0.1"
    Definir puerto
    Crear un socket para recibir mensajes
    Hacer que el socket escuche en la dirección y puerto definidos

Función manejar mensaje OSC (handle_osc_message):
    Si la dirección del mensaje es "/test":
        Imprimir mensaje recibido
```

```
        Cambiar tamaño de la esfera a (2, 2, 2)
        Si la esfera tiene un material:
            Cambiar color del material a rojo (1, 0, 0)
        Si no tiene material:
            Imprimir "No se encontró material en la esfera"

Función servidor OSC (osc_server):
    Iniciar el socket en la dirección y puerto definidos
    Mientras el servidor esté activo:
        Esperar recibir mensaje
        Decodificar mensaje
        Separar dirección y argumento del mensaje
        Llamar a la función manejar mensaje OSC con la dirección y argumento
recibidos
        Pausar brevemente para evitar sobrecargar el procesador

Función iniciar servidor OSC en un hilo (start_osc_thread):
    Crear un hilo que ejecute la función servidor OSC
    Iniciar el hilo

Ejecutar la función iniciar servidor OSC
Mantener el script en ejecución para que el servidor siga recibiendo
mensajes
```

Pseudocódigo inicial de Thonny (M5 Stack):

```
Iniciar conexión Wi-Fi:
    Intentar conectar a la red "GTDM" con contraseña "12345678" por 30
segundos
    Si la conexión es exitosa:
        Imprimir la IP asignada
    Si no se conecta:
        Imprimir "No conectado"

Definir servidor OSC:
    Dirección IP del servidor = "127.0.0.1" (localhost)
    Puerto de recepción OSC = 9004
    Crear socket UDP para enviar mensajes OSC

Configurar botón A:
    Pin 39 como entrada con resistencia pull-up
    Inicializar variable de estado del botón como "None"

Función para enviar mensaje OSC (send_osc_message):
    Crear mensaje OSC con la dirección y el mensaje
    Enviar mensaje OSC al servidor en la IP y puerto definidos
    Imprimir el mensaje enviado
```

```
Función para verificar el estado del botón A (check_button):
    Detectar si el botón A está presionado (current_state)
    Si el botón es presionado por primera vez:
        Imprimir "Botón A presionado"
        Llamar a la función para enviar mensaje OSC "/test mensaje de
prueba"
        Actualizar el estado del botón a "presionado"
    Si el botón es soltado:
        Actualizar el estado del botón a "liberado"

Bucle principal:
    Ejecutar la función para verificar el estado del botón
    Esperar 0.1 segundos para evitar lecturas excesivas
```

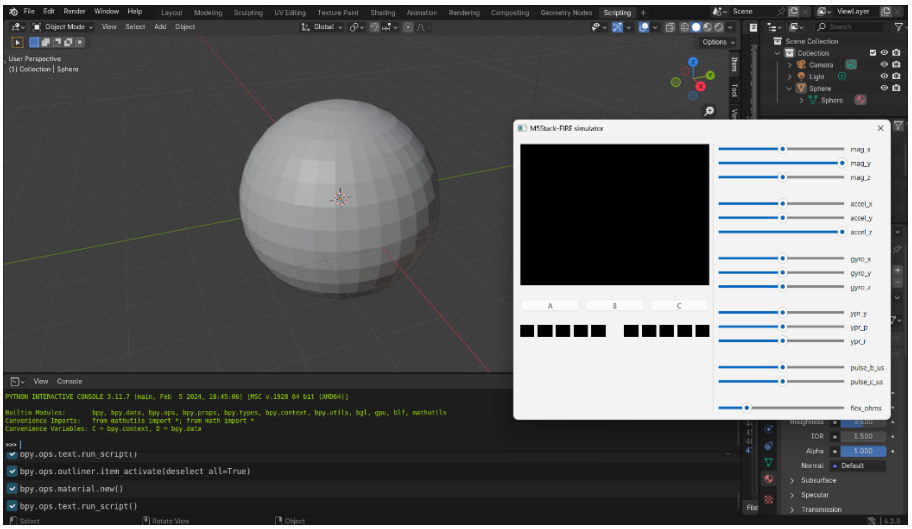


Figura 6. Estado inicial sin pulsar ningún botón, solamente ejecutando ambos códigos.

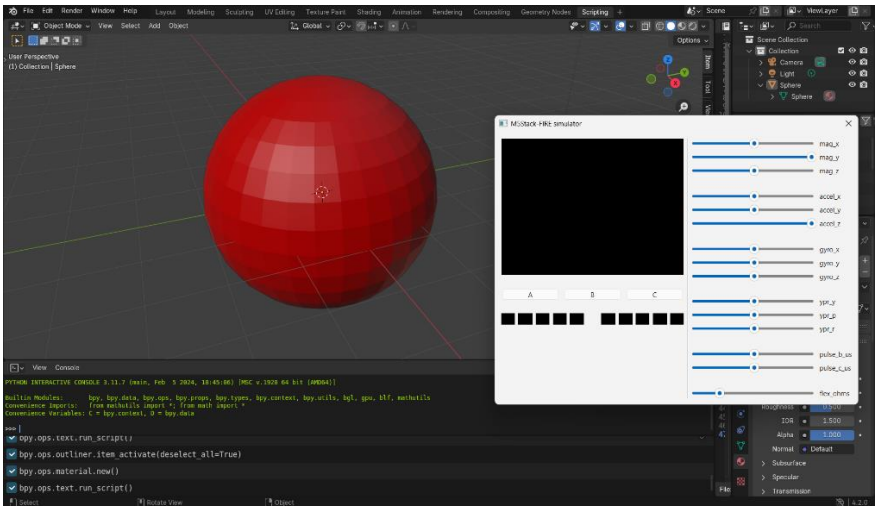


Figura 7. Estado tras pulsar el botón A, comprobación de que Blender recibe mensajes OSC.

Después de lograr que funcionara el botón A, se procedió a probar con el resto de los botones, B y C. Básicamente, se implementó la misma lógica utilizada para el botón A. Sin embargo, inicialmente pensamos que estos botones no funcionaban, hasta que descubrimos que el problema estaba relacionado con la configuración del puerto. El puerto utilizado tanto en el código del M5Stack como en el script de Blender debía ser cambiado para garantizar la recepción correcta de los mensajes. Tras corregir este detalle, lo botones B y C respondieron adecuadamente.

Para añadir un desafío adicional y mejorar la experiencia visual, decidimos cambiar el objeto inicial (una esfera) por uno más complejo: el modelo "Suzanne" (la cabeza de un mono incluida en Blender). Además, en el "modo color" (botón A), se concluyó que era demasiado básico que el objeto cambiara de color en su totalidad. Para añadir mayor realismo y complejidad, asignamos al objeto varios materiales diferentes, cada uno aplicado a un grupo de caras específicas. Al presionar el botón A, solo cambiaban de color aleatoriamente tres de los materiales: el de los ojos, la nariz y la boca, mientras que el resto permanecía sin modificar.

Cuando esta etapa demostró ser exitosa, se amplió el desarrollo para incluir múltiples funcionalidades asociadas a diferentes tipos de mensajes recibidos. Para cada dirección (address) específica – como `/color`, `/size`, `/rotate` y `/reset` – se implementaron acciones concretas que modificaban el estado del objeto dentro de la escena en Blender.

El mensaje `/color` se utilizó para cambiar dinámicamente el color del objeto. En esta etapa, se asignó un material al objeto, permitiendo que los cambios en su color fueran controlados por los valores recibidos a través del mensaje OSC. El mensaje `/size`, por su parte, permitió modificar el tamaño del objeto, ajustando sus valores de escala según los datos proporcionados. Además, el mensaje `/rotate` (*Figura 8*) habilitó la rotación del objeto en uno de sus ejes principales, simulando un movimiento fluido y controlado. Estas funcionalidades iniciales establecieron una base sólida para la interacción con el objeto.

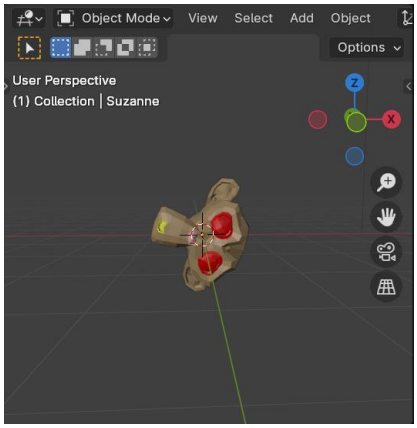


Figura 8. Comando /rotate

Con el fin de mejorar la experiencia visual y funcionalidad del sistema, se añadió un mensaje adicional: `/reset`. Esta funcionalidad devolvía el objeto a un estado inicial predeterminado, restableciendo su tamaño, posición, rotación y colores asignados. Este paso resultó importante para garantizar un punto de partida coherente cada vez que se realizaba una nueva acción sobre el objeto.

Sin embargo, surgieron problemas al intentar asignar la rotación del objeto al estado inicial `(0,0,0)`, ya que desconocíamos que en Blender era necesario utilizar Euler rotation para este propósito. Inicialmente, intentamos configurar la rotación con el siguiente código: `suzanne.rotation = (0,0,0)`.

Tras investigar más a fondo, descubrimos que la manera correcta de asignar la rotación inicial era utilizando `rotation_euler`, de la siguiente forma: `suzanne.rotation_euler = (0,0,0)`.

Durante las pruebas, se identificaron áreas para optimizar la interacción y realismo del sistema. Una de las mejoras clave fue la asignación de múltiples materiales a diferentes grupos de caras del objeto en lugar de utilizar un único material. Esto permitió que el objeto tuviera varias partes diferenciadas por color, lo que aumentó significativamente su realismo visual. Además, el mensaje `/color` (*Figura 9*) fue ajustado para modificar los colores de estas distintas secciones, en lugar de cambiar uniformemente todo el objeto.

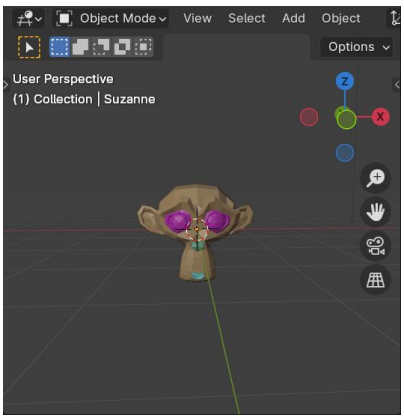


Figura 9. Comando /color

Otra mejora importante se implementó en la funcionalidad del mensaje `/size`. Además de cambiar el tamaño del objeto, se añadió un modificador de subdivisión (*Subdivision Surface*) cuando el objeto aumentaba de tamaño (*Figura 10*), lo que le otorgaba una apariencia más suavizada. Si el objeto disminuía de tamaño, el modificador se eliminaba automáticamente (*Figura 11*). Este ajuste dinámico contribuyó a un comportamiento más realista y atractivo visualmente.

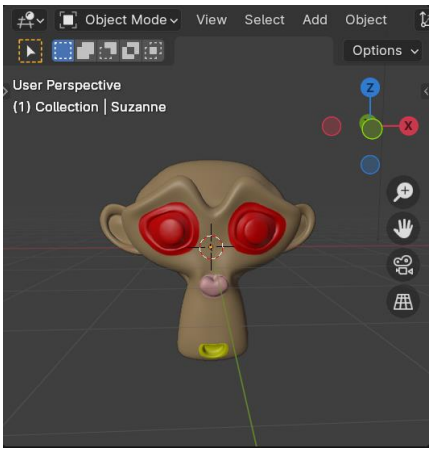


Figura 10. Comando /size (objeto aumentado)

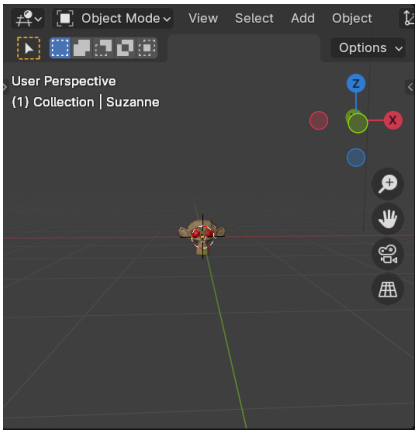


Figura 11. Comando /size (objeto disminuido)

Finalmente, se realizaron ajustes en la funcionalidad del mensaje `/reset` (*Figura 12*) para garantizar que no solo se restablecieran el tamaño, posición y rotación del objeto, sino también sus materiales y modificadores.

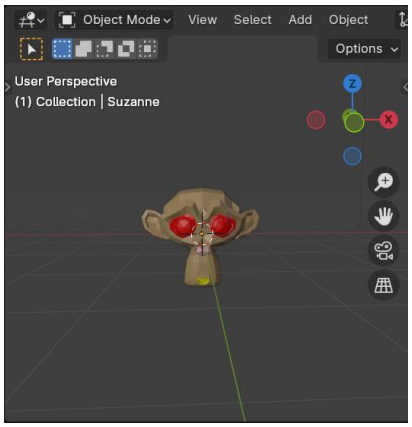


Figura 12. Comando /reset

Este proceso fue iterativo, y varios problemas surgieron durante su desarrollo, como la imposibilidad de ejecutar el script debido a conflictos en el puerto de comunicación (`[WinError 10048] Solo se permite un uso de cada dirección de socket (protocolo/dirección de red/puerto)`). Este problema se resolvió cambiando el puerto configurado para la conexión OSC, lo que permitió que el programa funcionara correctamente.

En definitiva, el desarrollo en Blender representó un desafío técnico que involucró tanto la integración de herramientas externas como la implementación de funcionalidades avanzadas. Cada etapa, desde la configuración inicial hasta la optimización de los detalles más pequeños, fue cuidadosamente diseñada y probada para garantizar un sistema fluido y visualmente atractivo. Las soluciones implementadas no solo resolvieron problemas técnicos, sino que también elevaron el nivel de interacción y realismo en la comunicación entre Blender, M5Stack y TouchOSC.

Pseudocódigo:

```
# Definir función para manejar mensajes OSC
Definir handle_osc_message(dirección, argumentos):
    Obtener objeto "Suzanne" en Blender
    Si el objeto no existe:
        Imprimir "No se encontró el objeto Suzanne"
        Salir de la función

    Si la dirección es "/reset":
        Imprimir "Reseteando objeto"
        Restablecer posición, escala y rotación del objeto a valores
predeterminados
        Si existe un modificador llamado "SUBSURF":
            Eliminar el modificador
            Imprimir "Modificador eliminado"
        Restaurar colores de los materiales a valores predeterminados

    Si la dirección es "/color":
        Imprimir "Cambiando color"
        Convertir el argumento en valor de yaw
        Calcular nuevos colores basados en yaw
        Asignar los colores calculados a los materiales correspondientes

    Si la dirección es "/size":
        Imprimir "Cambiando tamaño"
        Convertir el argumento en valor de roll
        Calcular factor de escala basado en roll, asegurándose de que esté
entre 0.1 y 3
        Ajustar la escala del objeto según el factor calculado
```

```
        Si el factor de escala es mayor que 1:
            Si el modificador "SUBSURF" no existe:
                Crear un modificador "SUBSURF"
                Imprimir "Modificador Subsurf añadido"
            Calcular nivel de subdivisión basado en roll
            Asignar nivel de subdivisión al modificador
            Imprimir "Nivel de subdivisiones ajustado"
        Si el factor de escala es menor o igual a 1:
            Si el modificador "SUBSURF" existe:
                Eliminar el modificador
                Imprimir "Modificador Subsurf eliminado"

    Si la dirección es "/rotate":
        Imprimir "Rotando objeto"
        Convertir el argumento en valor de pitch
        Ajustar rotación del objeto en el eje Y según pitch

# Definir servidor OSC
Definir osc_server():
    Configurar dirección e IP del servidor
    Crear y enlazar un socket UDP para recibir mensajes OSC
    Imprimir "Servidor OSC iniciado en dirección:puerto"
    Mientras el servidor esté activo:
        Recibir mensaje OSC del cliente
        Decodificar el mensaje en dirección y argumentos
        Llamar a handle_osc_message con la dirección y los argumentos
        Esperar 0.1 segundos

# Iniciar servidor OSC en un hilo separado
Definir start_osc_thread():
    Crear un hilo para osc_server
    Configurar el hilo como daemon
    Iniciar el hilo

# Iniciar la ejecución
Llamar a start_osc_thread()
```

3.4. Ejecución

El proceso de ejecución del programa en el entorno físico comenzó con la configuración y orden de inicio de las herramientas involucradas: Blender, TouchOSC y el entorno de desarrollo Thonny para el M5Stack. Aunque el orden de inicio entre Blender y TouchOSC no era estrictamente relevante, se estableció que ambos debían ejecutarse antes del programa de Thonny para garantizar la correcta conexión de los mensajes OSC con el dispositivo M5Stack. Esta secuencia fue

importante para evitar conflictos en las comunicaciones y garantizar una sincronización adecuada entre los componentes.

Al intentar ejecutar el programa por primera vez, en el entorno físico, surgieron problemas relacionados con la conexión a la red WiFi (*Figura 13*). Fue necesario conectar el portátil a la red GTDM del laboratorio, y una vez establecido el enlace, se utilizó el comando `ipconfig` en el CMD de Windows para identificar la dirección IPv4 del PC (*Figura 14*). Esta dirección IP se actualizó en el código de MicroPython del M5Stack. En el script de Blender se realizó un ajuste en la configuración del host, al igual que en TouchOSC, cambiando la dirección de `127.0.0.1` a `0.0.0.0` para permitir la comunicación a través de cualquier interfaz de red activa, ampliando así la compatibilidad.

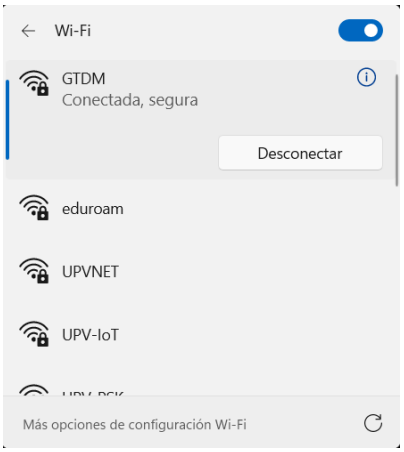
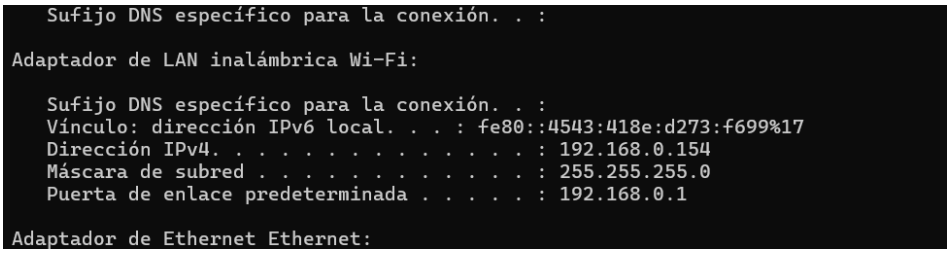


Figura 13. Conexión a la red WiFi



```
   Sufijo DNS específico para la conexión. . :

Adaptador de LAN inalámbrica Wi-Fi:

   Sufijo DNS específico para la conexión. . :
   Vínculo: dirección IPv6 local. . . : fe80::4543:418e:d273:f699%17
   Dirección IPv4. . . . . . . . . . . . . : 192.168.0.154
   Máscara de subred . . . . . . . . . . . : 255.255.255.0
   Puerta de enlace predeterminada . . . . . : 192.168.0.1

Adaptador de Ethernet Ethernet:
```

Figura 14. Dirección IPv4

Otro obstáculo importante surgió con la carpeta `uosc`, utilizada para manejar la comunicación OSC en el dispositivo M5Stack. Inicialmente, el dispositivo no era compatible con la última versión de la librería disponible en GitHub. Se intentó resolver este problema actualizando el firmware del M5Stack a una versión anterior, pero sin éxito. Finalmente, la solución consistió en usar una versión más antigua de `uosc`, proporcionada en los recursos del apartado de prácticas en PoliformaT. Esta

versión permitió una correcta integración entre el M5Stack y los demás componentes del sistema.

Durante la ejecución del código, se identificaron limitaciones en el manejo de cadenas de formato en el M5Stack, que no soporta el uso de cadenas de formato `f-strings`. Para resolver este inconveniente, se optó por utilizar el método clásico de formato de cadenas, como `"{} {}".format(address, message)`. Esto garantizó que el programa se ejecutara sin errores de sintaxis relacionados con el formato de los mensajes enviados.

Un error específico en el código MicroPython también generó problemas durante la ejecución, en particular en la línea 90, que manejaba la obtención de valores del IMU. Inicialmente, se intentó acceder a los valores de yaw, `pitch` y roll mediante un método privado (`mpu._get_ypr`). Sin embargo, esto generaba conflictos, por lo que se sustituyó por la utilización directa de `mpu.ypr`, una implementación más adecuada y compatible con el dispositivo. Este cambio fue fundamental para permitir la lectura fluida de los datos de orientación del IMU.

Tras resolver todos estos problemas técnicos, finalmente se logró a ejecutar el programa sin inconvenientes, y el sistema funcionó como se esperaba en general. Sin embargo, durante la visualización de los valores de yaw en el *fader* de TouchOSC, se identificaron limitaciones inherentes a la naturaleza de estos datos. A diferencia de pitch y roll, que se limitan dentro de un rango fijo de grados, el valor de yaw aumenta continuamente sin un límite superior claro, volviendo a cero después de alcanzar su valor máximo. Este comportamiento complicó la normalización de los datos entre 0 y 1 para representarlos correctamente en el *fader* de TouchOSC. Como resultado, la visualización de yaw mostró inconsistencias, ya que el paso entre valores no era gradual ni reflejaba el comportamiento esperado.

A fin de cuentas, la ejecución física del programa implicó superar una serie de desafíos técnicos, desde configuraciones red y compatibilidad de librerías hasta problemas con el formato de código y datos del IMU. Aunque se lograron solucionar la mayoría de los inconvenientes, algunas limitaciones inherentes al hardware y a los datos generados por el yaw del IMU persistieron, afectando parcialmente la experiencia visual en TouchOSC. Estos desafíos resaltaron la importancia de la flexibilidad y adaptabilidad en el desarrollo de sistemas de integración hardware-software.

4. CONCLUSIONES

Realizar este proyecto ha sido una experiencia extremadamente enriquecedora, ya que ha permitido aplicar de manera práctica los conceptos vistos en clase, llevándolos un paso más allá de lo esperado. Sin embargo, el mayor desafío ha sido enfrentarnos al protocolo OSC sin haber tenido previamente la práctica que estaba

programada en el curso. Esta ausencia de guía inicial nos empujó a movernos en un terreno desconocido y explorar herramientas y enfoques completamente nuevos por nuestra cuenta.

La integración de Blender y el protocolo OSC, aunque no formaron parte de las prácticas entregadas, nos motivó enormemente. Nos ofrecieron la oportunidad de extender lo aprendido en clase, y eso nos permitió ser creativas y proactivas en el diseño de soluciones. A pesar de que ya habíamos trabajado con el M5Stack y MicroPython anteriormente, la combinación de estos conocimientos con Blender representó un reto único.

Blender, una herramienta que conocimos el año pasado en la asignatura de Diseño Gráfico, ya nos resultaba interesante como software de modelado 3D. Pero la idea de interactuar con él de manera remota, sin siquiera tocar su interfaz, nos apasionaba aún más. Este desafío técnico, aunque intimidante al principio, nos impulsó a explorar nuevas formas de conectar software y hardware. Fue especialmente gratificante descubrir cómo traducir los datos de los sensores del M5Stack en movimientos y transformaciones visibles dentro de Blender.

Uno de los principales logros del proyecto ha sido hacer que el giroscopio del M5Stack envíe datos precisos y fluidos para controlar las transformaciones del objeto en Blender en tiempo real. Esto va más allá de enviar datos puntuales o usar botones, que es algo relativamente más sencillo. Lograr que los sensores respondieran dinámicamente y permitieran cambios continuos en función de los movimientos físicos fue un gran desafío. Normalizar estos valores para que las transformaciones fueran suaves y manejables implicó varias iteraciones de pruebas y ajustes, pero finalmente conseguimos un sistema estable y funcional. También desarrollamos funcionalidades adicionales, como un comando para reiniciar el estado del objeto, lo que hizo que el sistema fuera más práctico y fácil de usar.

En cuanto a las herramientas empleadas, trabajar con Blender y NodeOSC nos permitió explorar de manera práctica cómo conectar un entorno de diseño 3D con datos en tiempo real provenientes de un dispositivo externo. Aunque inicialmente habíamos considerado que TouchOSC desempeñaría un papel más importante, terminó siendo un visor complementario. Este ajuste en los planes nos hizo reflexionar sobre la necesidad de ser flexibles y de adaptar las ideas originales a las limitaciones técnicas y a las herramientas que mejor se ajusten a las necesidades del proyecto.

El desarrollo del proyecto no estuvo exento de desafíos técnicos. Por ejemplo, la falta de compatibilidad del M5Stack con algunas funciones modernas de Python nos obligó a buscar soluciones alternativas para implementar nuestro código. También experimentamos problemas recurrentes con los puertos de comunicación y algunas caídas de conexión inesperadas, lo que retrasó el progreso en varios

momentos. Sin embargo, cada obstáculo fue una oportunidad para aprender a resolver problemas de manera eficiente, fortalecer nuestras habilidades de programación y ampliar nuestra capacidad de análisis técnico.

Más allá de los aspectos técnicos, este proyecto también nos enseñó la importancia de la planificación, la paciencia y la capacidad de adaptación. Aprendimos a valorar el proceso tanto como el resultado final, ya que cada iteración nos permitió descubrir algo nuevo y mejorar la calidad del sistema. Nos dimos cuenta de que proyectos como este no solo requieren habilidades técnicas, sino también creatividad, perseverancia y trabajo en equipo.

En general, estamos muy satisfechas con los resultados obtenidos. Aunque sabemos que aún hay margen para mejoras, como añadir más opciones de interacción o perfeccionar la precisión de los sensores, sentimos que hemos creado una base sólida sobre la que podríamos construir futuros proyectos. Este trabajo no solo nos ayudó a consolidar los conocimientos adquiridos en clase, sino que también nos permitió explorar nuevas perspectivas creativas, como la idea de personalización interactiva que vemos en algunos videojuegos. Ese concepto nos inspiró desde el inicio y nos ayudó a visualizar cómo una interacción de tal calibre podría evolucionar hacia proyectos más grandes y complejos en el futuro.

En resumen, este proyecto nos permitió aplicar lo aprendido, explorar herramientas nuevas, y, sobre todo, disfrutar de un proceso creativo y desafiante que nos preparará mejor para futuros retos en nuestra formación y carrera profesional.